

Online Store & Courier Integration

A practical implementation guide for the existing Shipday backend

Recommendation: build one reusable shipment-quotation core, then expose it through separately authenticated APIs for online stores, courier providers, and Bobgo. Keep provider-specific formats at the edges; persist one canonical shipment and pricing model inside Shipday.

What the project already has

- Express API: `server.js` mounts routes under `/api`; `routes/index.js` currently mounts `/shipments` and `/pricing`.
- Shipment model: `models/Shipment.js` already stores waybill identifiers, `numberOfBoxes`, `parcel` arrays, `marketplaceName`, and `pricingBreakdown`.
- Pricing: `models/Pricing.js` and `utils/shipmentPricing.js` already calculate service pricing, packaging, and inter-province fees. `utils/packagingPricing.js` accepts multiple dimension entries.
- Shipment creation: `controller/admin.js` creates shipments and reuses the shared pricing helper. Existing shipment endpoints are not yet a partner-integration API contract.
- Courier/Bobgo integrations: the repository has no courier-provider or Bobgo adapter implementation identified; treat these as new modules, not extensions to the local Delivery model.

Keep the API roles distinct

- Online-store integration: a merchant asks for rates, chooses a service, creates a shipment/waybill, and adds parcels.
- Courier integration: Shipday calls an external carrier to quote, book, cancel, label, and track a shipment.
- Third-party courier API: an external courier calls Shipday to quote or accept Shipday-originated delivery work and sends status events back.
- Bobgo: a named marketplace/platform adapter presents available courier services and prices and handles its chosen-carrier flow.

Minimum architecture additions

Add versioned route modules, a partner API-key middleware and credential model, a shared quote/booking service, and provider adapters under `services/`. Keep user JWT authentication for existing app users; issue separate scoped credentials for integrations. Never make provider credentials or partner secrets part of a Shipment response.

1. Online Store Integration API

Store-facing quotes, bookings, multi-box waybills, and surcharges

Suggested merchant workflow

- 1. Quote: POST /api/v1/quotes with origin, destination, service preference, and a parcels[] list. Return itemized prices, available services, currency, and quote expiry.
- 2. Book: POST /api/v1/shipments with quoteId, merchant order reference, recipient/pickup details, and the final parcel list. Revalidate price and availability server-side.
- 3. Add boxes: POST /api/v1/shipments/{shipmentId}/parcels with one or more additional parcel records. Recalculate and return the updated waybill total.
- 4. Track/manage: GET /api/v1/shipments/{shipmentId}; add cancellation only when the shipment/provider status permits it.

```
POST /api/v1/quotes
{
  "serviceType": "express",
  "origin": { "postalCode": "...", "province": "..." },
  "destination": { "postalCode": "...", "province": "..." },
  "parcels": [{ "length": 40, "width": 30, "height": 20, "weight": 8 }]
}
```

Waybill and multi-box pricing

- Canonical parcel list: use parcels[] as the source of truth for package count and dimensions. Validate it against legacy numberOfBoxes during a migration period; reject mismatches rather than silently charging a different count.
- Large-box surcharge: add admin-configured oversize rules (measurement basis/threshold and fee) to Pricing and calculate per qualifying parcel. Confirm the business threshold and whether the charge is per box before implementation.
- Additional-box fee: configure an included parcel allowance and a fee for each additional parcel on one waybill. Calculate $\max(0, \text{parcelCount} - \text{includedCount}) \times \text{fee}$; retain a per-parcel/per-fee breakdown for audit and refunds.
- Persist quote facts: store quoteId, expiry, pricing version/snapshot, currency, parcel count, and itemized totals on booking. Do not recompute old paid waybills using later admin pricing.
- Money handling: use integer minor units (or the project's consistent rounded-money helpers), validate finite non-negative dimensions/weights, and test boundary values.

Reuse rather than duplicate

Extend utils/shipmentPricing.js (and its tests) to calculate per-parcel base rates, oversize fees, extra-box fees, packaging, and existing inter-province charges. Make both the quote endpoint and controller/admin.js shipment creation call the same service. Expand Shipment.pricingBreakdown to preserve each line item and the final total.

2. Courier Integrations

One internal contract; separate adapters for each carrier

Outbound carrier adapter pattern

Create a stable internal interface such as `quote()`, `book()`, `cancel()`, `createLabel()`, `track()`, and `parseWebhook()`. Implement one adapter per courier. Convert Shipday's canonical addresses/parcels into provider requests and normalize provider responses into a common `service/rate/label/status` shape.

- Suggested modules: `services/couriers/CourierAdapter.js` (contract), one provider file per carrier, and a shared coordinator that selects the provider and persists booking results.
- Persist external references: add a `CourierBooking` record linked to `shipmentId` with provider, service, external booking/consignment ID, provider status, label reference, timestamps, and sanitized error details.
- Reliability: use an idempotency key for booking; retry transient failures with limits; do not retry non-idempotent booking blindly. Save provider response IDs before reporting success.
- Status updates: map provider statuses into an explicit Shipday status map. Verify signed webhook payloads, deduplicate provider event IDs, and retain an event/audit record.
- Secrets: store provider credentials outside source control; encrypt at rest where applicable, restrict access, redact logs, and support credential rotation per tenant/account.

Third-party courier API is the reverse direction

Do not confuse Shipday's carrier adapters with an API used by outside couriers. Give each courier partner a scoped API key and its own contract, for example:

```
POST /api/v1/courier/quotes
POST /api/v1/courier/shipments/{shipmentId}/accept
POST /api/v1/courier/shipments/{shipmentId}/events
GET  /api/v1/courier/shipments/{shipmentId}
```

- Access rules: scope keys by partner and allowed operations; verify each courier can access only its assigned shipments; include request IDs, rate limits, and key revocation.
- Callbacks: accept signed, idempotent status events and validate legal status transitions before updating Shipment.

Fit with today's code

Keep provider booking data out of `models/Delivery.js`, which represents an internal delivery/driver workflow. Link any carrier booking to Shipment using a separate model so local fleet deliveries remain compatible.

3. Bobgo and Competitive Rates

Implement as a platform-specific adapter after confirming its API contract

Bobgo flow to support

- Rate discovery: translate a Bobgo quote request into Shipday's canonical quote input; request all eligible Shipday/carrier services and return normalized service names, delivery estimates, and prices in Bobgo's required format.
- Favorite courier: store the merchant's preferred service/provider mapping per Bobgo account. Return the preferred option clearly while still including comparable eligible rates when allowed by Bobgo's product contract.
- Booking: when Bobgo selects a service, book that exact service, save Bobgo's order reference and the provider's booking reference, and return the tracking/label information Bobgo expects.
- Tracking: map carrier webhooks or polling updates into Shipday status events and send the normalized update to Bobgo if its API supports callbacks.

Build the Bobgo adapter at the boundary

Add `services/marketplaces/bobgoAdapter.js` to map Bobgo payloads to the internal quote and booking services. Keep Bobgo-specific authentication, field names, response formatting, and error mapping inside that adapter. Avoid hard-coding Bobgo assumptions into Shipment or the pricing calculator.

Confirm before coding

- API documentation: obtain Bobgo's current partner API specification, authentication method, sandbox, webhook rules, and certification requirements.
- Rate semantics: confirm whether Bobgo expects Shipday retail prices, carrier net rates, markups, taxes, or currency conversion, and who owns the favorite-courier preference.
- Failure cases: agree how expired rates, unavailable favorites, duplicate bookings, partial multi-box bookings, cancellations, and refunds are represented.

Do not assume endpoint names or capabilities until Bobgo confirms them. The adapter boundary makes those changes local when the real contract is available.

Commercial controls

Define whether partner rates are public, account-specific, or contract-specific. Apply customer/partner pricing policy after carrier rates are normalized; record the chosen rate and its source on the booking for reconciliation.

4. Delivery Plan & Acceptance Checks

Small, testable increments that reuse the current backend

Suggested build order

- Phase 1 — shared pricing: add parcel-level pricing lines, oversize configuration, extra-box rules, price snapshots, and unit tests. Confirm thresholds and fees with the business owner.
- Phase 2 — merchant API: add versioned partner auth, quote and booking endpoints, parcel-add flow, input validation, idempotency, and API documentation.
- Phase 3 — first carrier: implement one real courier adapter end-to-end in sandbox (quote, book, label, tracking, cancellation where supported). Validate persistence and webhook signatures.
- Phase 4 — courier partner API: add scoped external-courier access, assignment checks, accept/update events, event deduplication, and partner-facing docs.
- Phase 5 — Bobgo: implement the confirmed contract using the marketplace adapter; test favorite selection and comparison-rate display with Bobgo sandbox credentials.

Acceptance tests

- Pricing: single parcel unchanged from current price; exactly-at-threshold and just-over-threshold box; one included box vs. additional boxes; multi-box and mixed oversize; no negative/NaN inputs; deterministic totals.
- Quote-to-book: valid unexpired quote succeeds; expired/changed quote returns a clear conflict or requote requirement; repeated idempotency key creates one waybill.
- Carrier failures: timeout and provider rejection do not create a false successful booking; safe retry does not duplicate a consignment; webhook replay does not duplicate status effects.
- Authorization: missing, revoked, wrong-scope, and cross-merchant credentials are rejected; one partner cannot retrieve another partner's shipments.
- Compatibility: existing app shipment creation, internal driver deliveries, waybill downloads, and current pricing tests continue working.

Project touchpoints

Start with `routes/index.js`, `routes/pricingRoutes.js`, `routes/shipments.js`, `controller/admin.js`, `models/Pricing.js`, `models/Shipment.js`, and `utils/shipmentPricing.js`. Add new isolated route, middleware, model, and adapter modules rather than putting all integration logic into the existing admin controller. The current test suite has pricing-focused tests under `tests/`.